

Lab Report – EE-170L Computer Programming (Lab 08)

NAME: _MAAZ WAHAB _____

REG NO: _BF25NWELE0719 _____

SEMESTER: 2nd

LAB: 8 (Programming)

DATE: __28 APRIL 2026 _____

1. Objective

The purpose of this lab is to help students understand and apply the following concepts:

- **Two-Dimensional Array**
- **Declaration and Initialization** of 2D arrays
- **Accessing elements** using row and column indices
- Performing operations such as **sum of rows and columns** and **finding maximum value** in a 2D array

2. Theory

A **two-dimensional array** is essentially a list of one-dimensional arrays. It can be visualized as a table with rows and columns. Each element is accessed using two indices:

$$a[i][j]$$

where i is the row index and j is the column index.

- **Declaration:**

cpp

```
int matrix[3][3];
```

- **Initialization:**

cpp

```
int matrix[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} };
```

- **Accessing Elements:**

cpp

```
cout << matrix[1][2]; // prints element at row 1, column 2
```

3. Programs & Outputs

Program 1: Declaration, Initialization, and Display of a 3x3 Matrix

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    // Declare and initialize a 3x3 matrix
```

```
    int matrix[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} };
```

```
    // Display matrix elements using nested loops
```

```
    cout << "Matrix Elements:" << endl;
```

```
    for(int i=0; i<3; i++) {    // Loop for rows
```

```
        for(int j=0; j<3; j++) {    // Loop for columns
```

```
            cout << matrix[i][j] << " ";
```

```
        }
```

```
    cout << endl;
```

```
}
```

```
    return 0;
}
```

Output:

Code

Matrix Elements:

1 2 3

4 5 6

7 8 9

Program 2: Sum of Rows and Columns

cpp

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    // Declare and initialize a 3x3 matrix
```

```
    int matrix[3][3] = { {2,4,6}, {1,3,5}, {7,8,9} };
```

```
    // Calculate sum of rows
```

```
    for(int i=0; i<3; i++) {
```

```
        int rowSum = 0;
```

```
        for(int j=0; j<3; j++) {
```

```
            rowSum += matrix[i][j];
```

```
        }
```

```
        cout << "Sum of Row " << i << " = " << rowSum << endl;
```

```
    }
```

```

// Calculate sum of columns
for(int j=0; j<3; j++) {
    int colSum = 0;
    for(int i=0; i<3; i++) {
        colSum += matrix[i][j];
    }
    cout << "Sum of Column " << j << " = " << colSum << endl;
}

return 0;
}

```

Sample Output:

Code

Sum of Row 0 = 12

Sum of Row 1 = 9

Sum of Row 2 = 24

Sum of Column 0 = 10

Sum of Column 1 = 15

Sum of Column 2 = 20

Program 3: Finding Maximum Value in a 4x4 Matrix

cpp

```
#include <iostream>
```

```
using namespace std;
```

```

int main() {

    // Declare and initialize a 4x4 matrix

    int matrix[4][4] = { {5,12,7,9}, {3,15,8,6}, {10,2,18,4}, {11,14,1,13} };


    // Assume first element is maximum

    int maxVal = matrix[0][0];


    // Traverse all elements to find maximum

    for(int i=0; i<4; i++) {

        for(int j=0; j<4; j++) {

            if(matrix[i][j] > maxVal) {

                maxVal = matrix[i][j];

            }

        }

    }


    cout << "Maximum Value in Matrix = " << maxVal << endl;

    return 0;

}

```

Output:

Code

Maximum Value in Matrix = 18

4. Discussion

- Two-dimensional arrays provide a structured way to store tabular data.
- Nested loops are essential for traversing rows and columns.

- Operations like **row/column sums** and **finding maximum values** demonstrate practical applications in data processing.

5. Conclusion

In this lab, students learned how to **declare, initialize, and access two-dimensional arrays** in C++. They also practiced performing operations such as **summing rows and columns** and **finding the maximum value**. These skills are fundamental for handling matrix-based problems in programming.